

[1. MeanFilterGray.cpp]

MeanFilterGray.cpp 는 이미지에 uniform mean filtering 을 적용하는 프로그램이다. uniform mean filtering 은 아래와 같이 레퍼런스 픽셀 값을 주변 픽셀들의 산술평균 값으로 설정해 노이즈를 제거하는 작업으로, 간단한 low-pass filter 이며 아래와 같이 수행한다.

$$O(i, j) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) I(i + s, j + t)$$

이때 filtering 하고자 하는 픽셀이 이미지의 모서리에 존재한다면 픽셀 값들을 mirroring 하거나 zero padding 하고, filter kernel을 조정하여 valid한 것만 사용할 수 있다.

- (1) mirroring
모서리의 픽셀값을 이미지 바깥의 filter kernel에 매핑하는 방식이다.
- (2) zero padding
이미지 범위를 벗어난 픽셀들에 0을 할당하는 방식이다.
- (3) filter kernel 조정
filter kernel을 조정하여 이미지 범위 내의 것들만 사용하는 방식이다.

uniform mean filter를 적용하면 픽셀들 간의 차이가 줄어들어 이미지가 blurry해지고, 마스크의 크기가 커질 수록 결과 이미지가 더욱 흐려진다. 이때 마스크의 크기가 커지면 연산량이 증가하므로, 프로그램 효율을 위해 separable filtering이 중요하다. associative property는 여러 linear function을 수행할 때 이들을 어떤 순서로 수행하든 결과가 동일한 성질로, 선형 필터인 uniform mean filtering 또한 associative property를 만족한다. $n \times n$ 필터를 사용할 경우 한 픽셀 당 n^2 번의 연산이 수행되지만, separable filtering을 통해 $n \times n$ 필터를 $n \times 1$ 필터와 $1 \times n$ 필터로 분리한다면 픽셀 당 $2n$ 번의 연산으로 동일한 결과를 얻을 수 있다. 즉, 여러 단계로 필터링할 때 필터의 순서를 변경하거나 필터를 결합 또는 분리할 수 있으며, 이는 runtime 측면에서 매우 유리하다.

1. main() 함수

namedWindow(), imshow() 함수를 사용해 원본 grayscale 이미지와 mean filtering 한 결과 이미지를 개별 창에 출력하며, 그 과정에서 meanfilter() 함수를 호출해 mean filtering 을 적용한 것을 output 에 저장한다. 위 코드에서는 $n=3$ 을 전달하고 있으므로, 커널 사이즈는 $2 \times 3 + 1 = 7$, kernel 행렬의 크기는 $7 \times 7 = 49$ 이다.

```
namedWindow("Grayscale", WINDOW_AUTOSIZE);
imshow("Grayscale", input_gray);

output = meanfilter(input_gray, 3, "mirroring"); //Boundary process: zero-paddle, mirroring,
adjustkernel
```

```
//Boundary process: zero-paddle, mirroring, adjustkernel
namedWindow("Mean Filter", WINDOW_AUTOSIZE);
imshow("Mean Filter", output);
```

2. 결과분석: 입력 이미지로 "lena.jpg"를 사용했다.

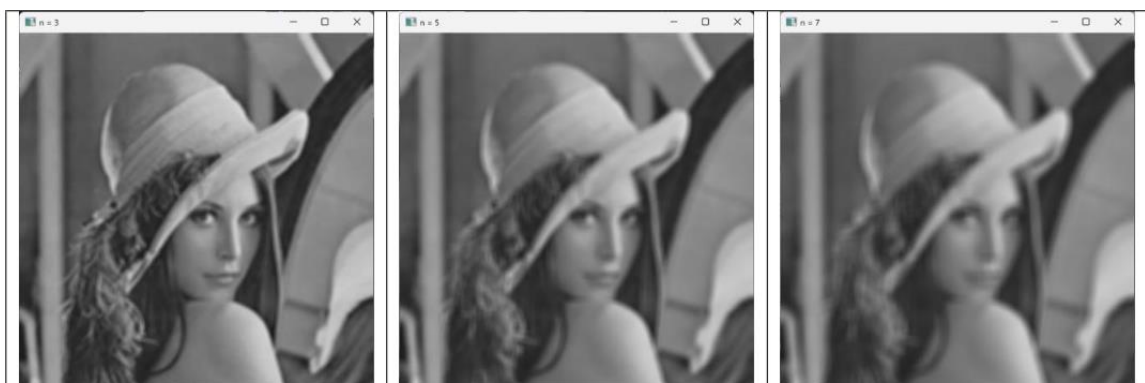
① boundary 처리 방식별 비교

모든 경우, uniform mean filtering을 거쳐 흐릿해진 결과가 출력되었다. zero padding의 경우 boundary 바깥의 픽셀을 0으로 채우므로 valid한 픽셀들로만 계산한 것보다 실제 적용되는 평균값이 낮아진다. 따라서 zero padding 결과 이미지의 모서리 부근이 어두워진 것을 확인할 수 있으며, mirroring과 adjust kernel 옵션에서는 이러한 효과가 관찰되지 않는다.



② 커널 크기별 비교

동일한 입력 이미지와 동일한 옵션(=mirroring)으로, n을 3, 5, 7로 증가시키며 3번 실행시켜 각각의 실행시간과 결과 이미지를 비교하였으며, 이를 위해 코드 일부를 수정하였다. n이 커질수록 실행 시간이 늘어나며, 결과 이미지가 더욱 blurry해지는 것을 확인할 수 있었다. n이 증가하며 커널 크기가 커지면 하나의 픽셀을 계산하기 위해 더 넓은 영역의 픽셀 값들을 평균하게 되고, 이 경우 세부 정보들이 손실되어 이미지가 부드럽고 흐릿하게 보인다.



(위 사진은 왼쪽부터 순서대로 n=3, n=5, n=7 일 때의 결과이다)

[2. MeanFilterRGB.cpp]

MeanFilterRGB.cpp는 이미지에 uniform mean filtering을 적용하며, 경계값 처리 옵션으로 mirroring, zero padding, adjust kernel을 제공하는 프로그램이다. 전체적인 코드 흐름은 MeanFilterGray.cpp와 유사하나 입력으로 컬러 이미지를 사용한다.

1. 코드분석
2. 결과분석
 - ① boundary 처리 방식별 비교
Grayscale 에서와 동일한 결과가 나타났다.



- ② 커널 크기별 비교

동일한 입력 이미지와 동일한 옵션(=mirroring)으로, n 을 3, 5, 7로 증가시키며 3번 실행시켜 각각의 실행시간과 결과 이미지를 비교하였으며, MeanFilterGray.cpp에서 수정한 것과 동일하게 코드 일부를 수정하였다. MeanFilterGray.cpp와 마찬가지로 n 이 커질수록 더 많은 세부 정보가 손실되어 이미지가 더욱 blurry해진다. 또, MeanFilterGray.cpp에서 동일하게 실행시켰을 때와 비교해 전반적인 코드 수행 시간이 증가하였는데, 컬러 이미지 처리 시 R, G, B에 대해 모두 연산하므로 연산량이 증가하여 MeanFilterGray.cpp에서 $n=3, n=5, n=7$ 일 때의 실행 시간이 각각 약 0.5초, 약 1.4초, 약 2.3초였던 것에 비해 약 2초, 약 4초, 약 8초가 소요되었다.

[3. GaussianGray.cpp]

GaussianGray.cpp는 이미지에 Gaussian filtering을 적용하며, 경계값 처리 옵션으로 mirroring, zero padding, adjust kernel을 제공하는 프로그램이다. Uniform mean filtering은 커널 내의 픽셀들을 단순 산술 평균한 값을 매핑하므로, 레퍼런스 픽셀 (i, j) 와 커널 픽셀의 거리로 인한 연관성을 반영하지 못한다. GaussianGray.cpp는 이미지에 gaussian filtering을 적용하는 프로그램으로, 레퍼런스 픽셀의 값을 주변 픽셀들의 가중 평균값으로 변경하여, 즉 픽셀 간 거리를 고려해 가중치를 부여함으로써 보다 자연스러운 filtering이 가능하게 한다. 이다. Gaussian 함수의 그래프는 좌우 대칭인 종 모양 곡선으로, |표준편차|가 증가할수록 값이 급격하게 감소하는 형태이다. 중심에서 멀어질수록 함수 값이 감소하므로 레퍼런스 픽셀과 가까운 픽셀에 더 큰 가중치를 부여하며, 표준편차(σ) 값이 커질수록 이미지가 더욱 흐려진다.

<결과분석>

- ① boundary 처리 방식별 비교

input = input_gray, n=1, sigmaT=1, sigmaS=1 로 각각 호출하였다.



모든 옵션에서 원본 이미지와 결과 이미지가 정상적으로 출력되었으며, zero padding 옵션의 경우 mean filtering에서와 마찬가지로 모서리 부근의 픽셀들이 어두워진 것을 확인할 수 있다.

② 커널 크기별 비교

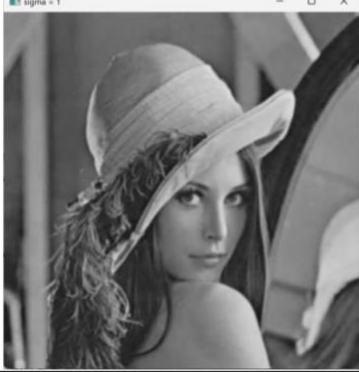
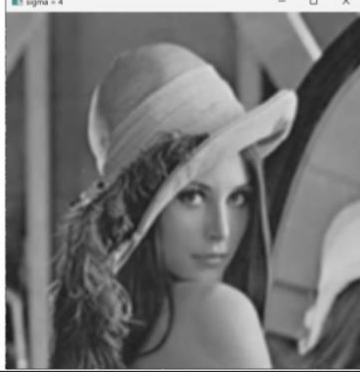
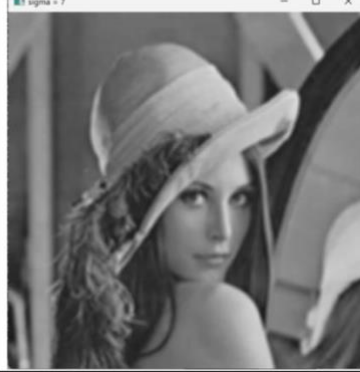
동일한 입력 이미지와 동일한 옵션(=mirroring)으로, n 을 3, 5, 7 로 증가시키며 3 번 실행시켜 각각의 실행시간과 결과 이미지를 비교하였다. MeanFilterGray.cpp 에서 수정한 것과 동일하게 코드 일부를 수정하였다. 코드 일부를 수정하였다.



n이 커질수록 실행 시간이 증가하지만 mean filtering에서 동일하게 실행했을 때보다 커널 크기에 따라blurry해지는 정도가 적다. n=20으로 극단적으로 증가시켜 실행했을 때도 육안으로 유의미한 차이는 드러나지 않는다. 왜냐하면, Gaussian 함수는 중심에서 최댓값을 가지며 지수 함수의 특성을 가진다. 따라서 중심에서 멀어질수록, 즉 레퍼런스 픽셀 (i, j)에서 멀리 떨어진 픽셀일수록 filtering 효과에 기여하는 정도가 급격히 줄어든다. n을 증가시켜도 중심 픽셀에 가까운 픽셀들만 blurring에 큰 영향을 미치므로 커진 커널이 모두 평균값 계산에 균등한 정도로 포함되던 mean filtering 보다 n의 크기가 blurring에 미치는 영향이 적기 때문이다.

③ 표준편차 크기별 비교

$n=3$, $opt="mirroring"$ 으로 통일하여 σ_T , σ_S 를 1, 4, 7로 증가시키며 실험해 보았다.

$\sigma = 1$	$\sigma = 4$	$\sigma = 7$
실행 시간: 0.869661 초	실행 시간: 0.985839 초	실행 시간: 0.89725 초
		

표준편차가 증가하며 결과 이미지가 blurry해지는 것을 확인할 수 있다.

④ σ_T , σ_S 를 anisotropic 하게 전달한 경우.

수직 방향과 수평 방향 표준편차가 다른 anisotropic 한 케이스를 테스트하기 위해 $\sigma_T=15$, $\sigma_S=1$ 인 경우와 $\sigma_T=1$, $\sigma_S=15$ 인 경우에 대해 결과 이미지를 비교하였으며, 이때 $n=5$, $opt="mirroring"$ 으로 통일하였다.



이전까지의 실험에서는 σ_T , σ_S 에 동일한 인자를 전달하여 isotropic 한 필터링을 적용했지만, 위의 사진들과 같이 수직과 수평방향의 표준편차를 다르게 설정하면 blur 효과가 한쪽 방향으로 강하게 나타남을 알 수 있었다. 수평 방향 표준편차인 σ_T 를 더 크게 하면 가로 방향으로 가중치 분포가 넓게 퍼져 가로로 blur 되며, 수직 방향 표준편차인 σ_S 를 더 크게 하면 세로로 blur 되는 것을 확인할 수 있다.

[4. GaussianRGB.cpp]

GaussianRGB.cpp 는 컬러 이미지에 Gaussian filtering 을 적용하는 프로그램이다. RGB 각 채널에 대해 따로 접근하고 연산하는 부분을 제외하면 함수 정의, boundary 처리 등 대부분의 코드가 GaussianGray.cpp 와 동일하게 작동한다.

1. 경계 픽셀 처리

옵션 별 인덱싱 등 구현 방법은 동일하지만, MeanFilterGray.cpp 코드에서 레퍼런스 픽셀 계산을 위해 $sum1 += kernelvalue * (float)(input.at<G>(i + a, j + b));$ 와 같이 (uniform 한 커널 값)*(이미지 픽셀값)으로 계산한 것과 달리 $sum1 += kernel.at<float>(a + n, b + n) * input.at<G>(i + a, j + b);$ 와 같이 각 픽셀의 커널 값을 사용한다. zero padding 옵션의 경우 위와 같으며, sum1 에 (커널*이미지 값) 결과를 합하는 코드를 제외하면 uniform mean filtering 의 코드와 동일한 것을 확인할 수 있다. 다른 옵션들도 마찬가지로 sum1 을 계산하는 부분 외에는 동일한 메커니즘을 가진다

```
if (!strcmp(opt, "zero-paddle")) {  
    float sum1 = 0.0;  
    for (int a = -n; a <= n; a++) { // 수직  
        for (int b = -n; b <= n; b++) { // 수평  
            if ((i + a <= row - 1) && (i + a >= 0) && (j + b <= col - 1) && (j + b >= 0)) {  
                sum1 += kernel.at<float>(a + n, b + n) * input.at<G>(i + a, j + b);  
            }  
        }  
    }  
    output.at<G>(i, j) = (G)sum1;  
}
```

<결과분석>

① boundary 처리 방식별 비교

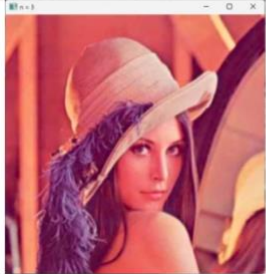
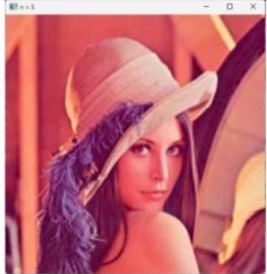
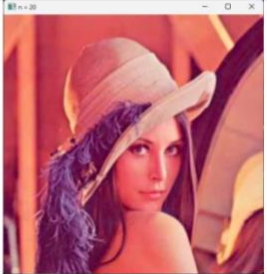
input=input, n=1, sigmaT=1, sigmaS=1 로 각각 호출하였다.



② 커널 크기별 비교

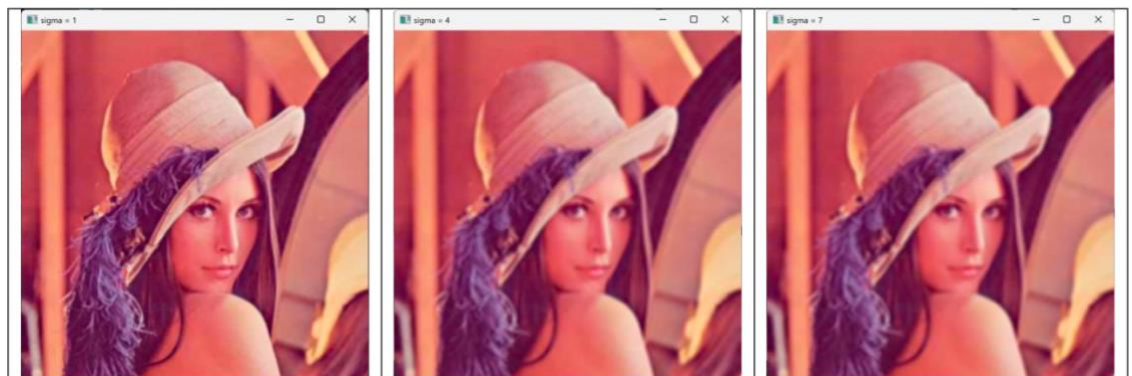
동일한 입력 이미지와 미러링 옵션으로 n=3,5,7 로 바뀌가면서 실행 시간과 결과 이미지를 비교했다.

blur 측면에서 큰 차이를 보이지는 않는다. 다만, RGB 각각에 대한 처리가 필요하기에, 연산량과 수행시간이 증가되었다.

n=3	n=5	n=7	n=20
실행 시간: 7.44536초	실행 시간: 7.44536초	실행 시간: 15.4009초	실행 시간: 121.508초
			

③ 표준편차 크기별 비교

$n=3$, $opt="mirroring"$ 으로 통일하여 σ_T , σ_S 를 1, 4, 7 로 바꾸면서 적용해보았다. 아래 그림 왼쪽부터 $\sigma=1$, $\sigma=4$, $\sigma=7$ 이고, 실행시간은 대략 3.6 초, 3.18 초, 3.24 초 였다. 표준편차가 커질수록(오른쪽으로 갈수록) 결과 이미지가 더 blurry 해지는 것을 관찰 할 수 있었다.



④ anisotropic 한 경우

수직 방향과 수평 방향 표준편차가 다른 anisotropic 한 케이스를 테스트하기 위해 $\sigma_T=15$, $\sigma_S=1$ 인 경우와 $\sigma_T=1$, $\sigma_S=15$ 인 경우에 대해 결과 이미지를 비교하였으며, 이때 $n=5$, $opt="mirroring"$ 으로 통일하였다.

결과를 보면 수평방향 표준편차가 클때는 가로로 블러 효과가 나타났고, 수직방향 표준편차가 클 경우에는 세로로 블러링이 나타났다.



[5. SobelGray.cpp]

SobelGray.cpp 는 grayscale 이미지에 sobel 필터를 적용해 원본 이미지와 결과 이미지를 출력하며, 경계값 처리 방식으로는 mirroring 만을 사용한다. Sobel 필터는 이미지의 경계(edge)를 검출할 때 사용하는 필터이다. 이미지의 경계는 이미지의 밝기가 급격하게 변화하는 부분으로, 이미지를 함수 $f(x, y)$ 로 표현할 때 f 의 1차 미분 값이 크게 나타나는 부분으로 정의할 수 있다.

1. 함수정의

input의 행과 열을 각각 col, row에 저장. 해당 크기의 Mat형 결과 이미지 객체 output을 선언한다. 변수 n에는 필터의 크기가 저장된다. 여기서는 3 by 3 커널을 사용하므로, n=1로 초기화한다.

```
int row = input.rows;
int col = input.cols;
int n = 1; // Sobel Filter Kernel N

// Initialiazing 2 Kernel Matrix with 3x3 size for Sx and Sy
int Sx[3][3] = { {-1, 0, 1}, {-2, 0, 2}, {-1, 0, 1} };
int Sy[3][3] = { {-1, -2, -1}, {0, 0, 0}, {1, 2, 1} };
```

2. 경계 처리 및 커널 연산

```
for (int i = 0; i < row; i++) {
    for (int j = 0; j < col; j++) {
        float Xsum = 0.0, Ysum = 0.0;

        for (int a = -n; a <= n; a++) {
            for (int b = -n; b <= n; b++) {
                // Use mirroring boundary process
                // Find output M(x,y) = sqrt( input.at<G>(x, y)*Sx + input.at<G>(x, y)*Sy )
```



```

        if (i + a > row - 1) {
            tempa = i - a;
        }
        else if (i + a < 0) {
            tempa = -(i + a);
        }
        else {
            tempa = i + a;
        }
        if (j + b > col - 1) {
            tempb = j - b;
        }
        else if (j + b < 0) {
            tempb = -(j + b);
        }
        else {
            tempb = j + b;
        }
Xsum += Sx[a + 1][b + 1] * input.at<G>(tempa, tempb);
Ysum += Sy[a + 1][b + 1] * input.at<G>(tempa, tempb);
}
} // end of for(-n~n)

int M = sqrt(Xsum * Xsum + Ysum * Ysum);
// 0~255
if (M < 0) { M = 0; }
if (M > 255) { M = 255; }

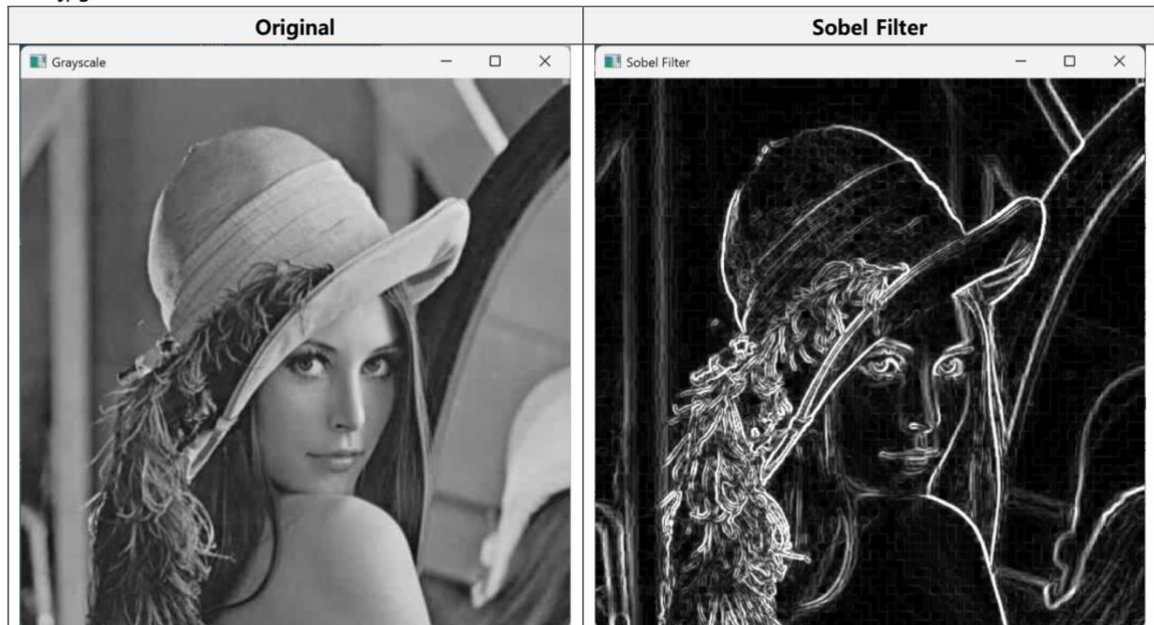
output.at<G>(i, j) = (G)M;

}

```

0 부터 row 까지, 0 부터 col 까지 2 중 for 문을 통해 모든 픽셀에 대해 계산하며, 매 내부 반복마다 수평 방향 커널 Sx 를 적용한 부분합을 저장할 Xsum, 수직 방향 커널 Sy 를 적용한 부분합을 저장할 Ysum 을 선언하고 0 으로 초기화한다. 레퍼런스 픽셀 (i, j) 에 대해 mirroring 방식으로 tempa 와 tempb 를 계산하고, 계산한 (tempa, tempb) 픽셀에 커널 값을 곱한 뒤 Sx 를 곱한 것은 Xsum 에, Sy 를 곱한 것은 Ysum 에 더한다. 이때 Sx[a][b]와 Sy[a][b]가 아닌 Sx[a+1][b+1]과 Sy[a+1][b+1]를 곱하는 것에 주의해야 하는데, 이는 커널 Sx와 Sy는 3*3 크기로 선언되어 0, 1, 2의 인덱스를 사용하는 반면 반복문에서는 커널 중심을 기준으로 -1, 0, 1 의 인덱스를 사용하고 있어 커널 값을 올바르게 매핑하기 위함이다. -n 부터 n 까지 반복하며 커널 내 픽셀들의 총합을 구한 뒤, 해당 커널의 Xsum 과 Ysum 을 이용하여 gradient magnitude M 을 계산한다. 이후 M 이 0~255 의 범위를 벗어났을 경우 해당 범위 내로 수정하고 output 의 (i, j) 위치에 계산한 M 을 매핑한다.

<결과분석>



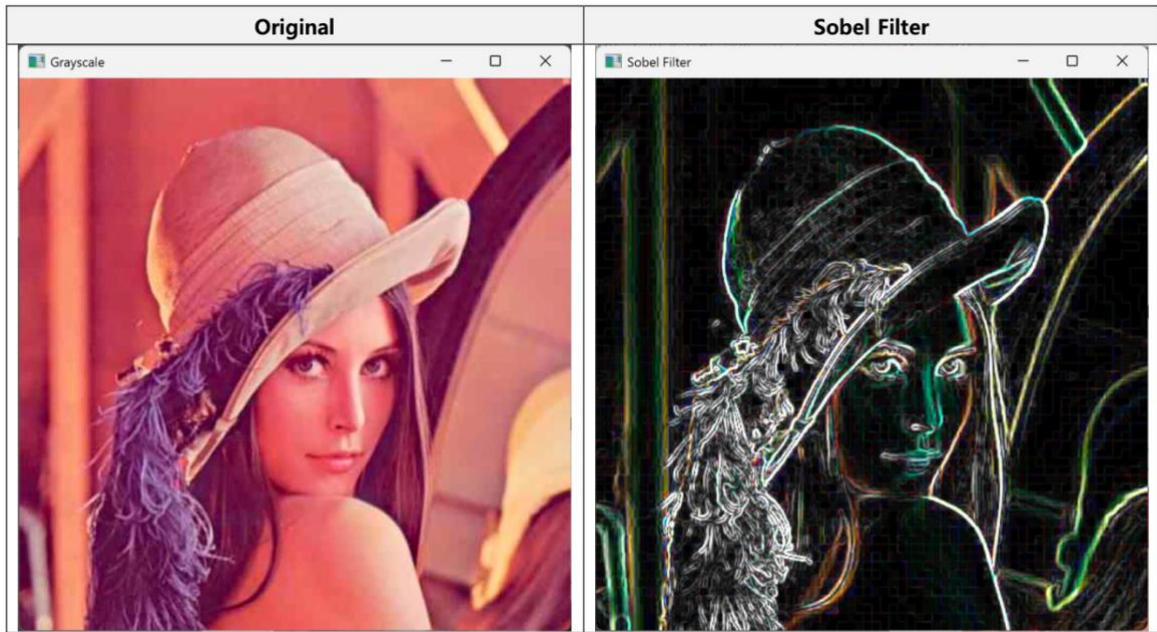
"lena.jpg" 이미지를 사용했다. 실행 사진을 보면 이미지의 경계가 검출된 것을 확인 할 수 있다. Sobel 필터는 이미지의 밝기 변화율(Gradient)을 계산하는 1차 미분 연산을 통해 급격한 밝기 차이가 발생하는 지점을 에지로 검출한다. 특히 커널 중심에 높은 가중치를 부여하는 가중치 평균(Weighted Average) 구조는 단순 차분보다 노이즈에 강인하며, 미세한 잡음을 억제하고 경계를 부드럽게 보존하는 Smoothing 효과를 동시에 수행한 것을 볼 수 있다.

[6. SobelRGB.cpp]

이 파일은 컬러 이미지에 대해 sobel 필터를 적용하는 프로그램이다. 이미지에 접근할 때, R,G,B 채널을 분리하며, 커널 내 총합도 채널별로 계산한다. 이것 이외에는 SobelGray.cpp 와 동일한 내용이다.

<결과분석>

lena.jpg 이미지의 경계가 검출되었으며, 컬러 이미지를 사용했으므로 결과 이미지에도 색상이 표시되어 보인다.



[7. LaplacianGray.cpp]

LaplacianGray.cpp 는 grayscale 이미지에 Laplacian 필터를 적용해 이미지 경계를 검출한다. Laplacian 필터는 2 차 도함수를 계산하며, 커널은 1 개만을 사용한다.

<결과분석>

라플라시안 필터는 이미지의 밝기 변화를 2 차 미분하여 에지를 검출한다. 1 차 미분 기반인 Sobel 필터가 수평과 수직방향을 각각 계산한 후 합치는 것과 달리, 라플라시안은 방향에 의존하지 않고 모든 방향의 에지를 한 번에 검출하는 등방성 성질을 가진다. 결과 이미지에서 방향에 관계없이 물체의 윤곽선이 가늘고 고르게 추출된 것은 이러한 특성이 반영된 결과다.

결과 사진에 에지가 매우 얇게 표현되면서도 배경에 미세한 잡음이 많이 섞여 보이는 이유는 2 차 미분의 높은 민감도 때문이다. 라플라시안 연산은 밝기가 미세하게 변하는 지점에서조차 결과값이 크게 튀기 때문에, 에지를 날카롭게 잡아내는 동시에 원본 이미지에 포함된 미세한 노이즈까지 과도하게 증폭시키는 경향이 있다.



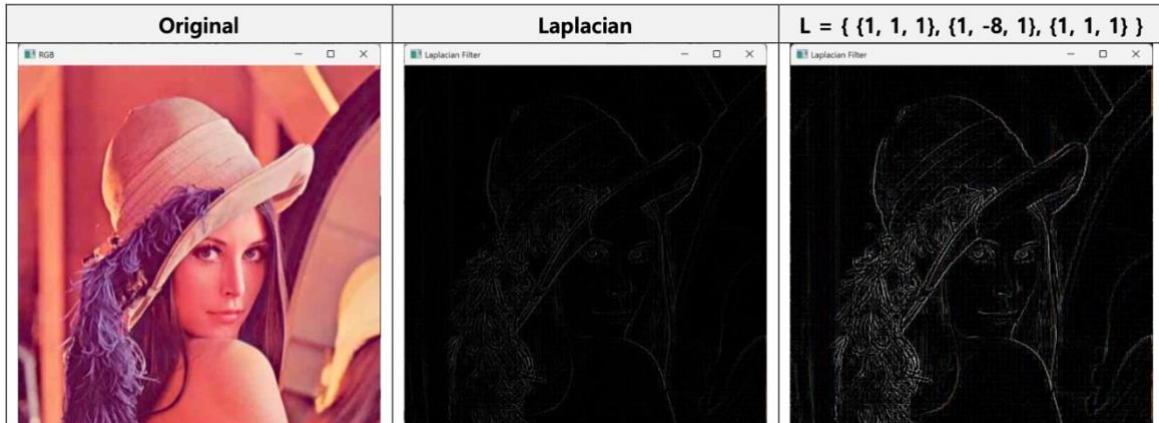
세번째 결과 사진은 $L = \{ \{1, 1, 1\}, \{1, -8, 1\}, \{1, 1, 1\} \}$ 로 커널을 조정한 결과이다. 더욱 선명해졌다.

[8. LaplacianRGB.cpp]

LaplacianRGB.cpp 는 컬러 이미지에 laplacian 필터를 적용한다. input 을 grayscale 로 변환하는 과정 없이 laplacianfilter() 함수에 컬러 input 을 그대로 전달하며, 이를 제외하면 main 함수는 LaplacianGray.cpp 와 동일하다.

<결과분석>

laplacian 필터링된 결과 이미지가 출력되며, 마찬가지로 $L = \{ \{1, 1, 1\}, \{1, -8, 1\}, \{1, 1, 1\} \}$ 로 커널을 조정했을 때 경계가 더 선명하게 검출된다.



[9. GaussianGray_sep.cpp]

GaussianGray_sep.cpp는 grayscale 이미지에 대해 gaussian 필터를 적용하되, GaussianGrap.cpp에서 $n \times n$ 커널을 적용했던 것과 달리 커널을 분리하여 $n \times 1$ 커널과 $1 \times n$ 커널을 각각 적용한다.

<결과분석>

input=input_gray, n=1, sigmaT=1, sigmaS=1 로 경계 처리 옵션을 달리하여 호출하였다.

그 결과 아래 결과 사진들과 같이 노이즈가 사라지고 흐려진 이미지가 되었음을 볼 수 있다.



[10. GaussianRGB_sep.cpp]

GaussianGray_sep.cpp 와 같이 $n \times 1, 1 \times n$ 크기로 분리한 2 개의 커널로 gaussian blurring 을 수행하되, 입력 이미지로 컬러 이미지를 사용한다.

1. Mat gaussianfilter(const Mat input, int n, float sigmaT, float sigmaS, const char* opt) 함수 GaussianGray.cpp 에서 N*N 커널을 1 번 적용하는 것과 달리 N*1 커널과 1*N 커널을 각각 적용하며, 각 적용마다 내부 구현은 인덱스 사용을 제외하면 GaussianGray.cpp 와 동일하다.

```
int row = input.rows;
int col = input.cols;
int kernel_size = (2 * n + 1);
int tempa;
int tempb;
float denomT, denomS;
// float kernelvalue;

// Initialiazing Kernel Matrix
Mat kernelT = Mat::zeros(kernel_size, 1, CV_32F); // N*1
Mat kernelS = Mat::zeros(1, kernel_size, CV_32F); // 1*N
```

<결과분석>

GaussianGray_sep.cpp 에서의 결과와 동일한 결과를 볼 수 있다. 다만 수행시간이 grayscale 일때에 비해 증가했음을 볼 수 있다.

GaussianRGB 또한 커널을 $n \times 1$, $1 \times n$ 으로 분리했을 때 분리하지 않은 것보다 수행 시간이 짧다. $n=10$, $\sigma_T=1, \sigma_S=1$, $opt="zero-paddle"$ 로 통일하여 GaussianRGB.cpp 와 GaussianRGB_sep 에서 $output = gaussianfilter(input, 10, 1, 1, "zero-paddle");$ 을 각각 호출하면, not separated 의 경우에는 33.33 seconds 가 출력 되는데에 반해, separable filter 일 경우에는 2.82 seconds 가 뜬다. RGB 3 개의 채널에 대해 모두 계산하므로 grayscale 이미지를 사용할 때보다 수행 시간이 증가했지만, 여전히 커널을 분리했을 때가 분리하지 않았을 때보다 월등히 빠르게 작동한다.



[11. UnsharpGray.cpp]

UnsharpGray.cpp는 grayscale 이미지에 대해 unsharp masking을 적용한다. Unsharp masking은 이미지에 low-pass 필터를 적용해 blur한 뒤, low-frequency 성분을 가진 blurry한 이미지와 high-frequency 요소를 포함한 원본 이미지를 비교해 강조하고 싶은 요소를 추출하며, 추출한 edge를 조절해 보다 선명해진(sharp) 결과 이미지를 생성한다. 아래는 해당 공식이다.

$$output = \frac{I - kL}{1 - k} \quad (I = image, L = low - pass image, k = scaling factor)$$

1. Mat unsharpmask

2 중 for 문으로 이미지 전체 픽셀을 순회하며 각 레퍼런스 픽셀 (i, j)에 대해 계산하는 것은 UnsharpGray.cpp 와 동일하나, 컬러 이미지를 사용하고 있으므로 output 변수 o 를 RGB 채널을 분리한 o_r, o_g, o_b 로 사용하고 input, temp, output 에 접근할 때도 output.at<C>(i, j)[0] =(G)(o_r); 과 같이 [0], [1], [2] 채널을 구분해 채널별로 각각 계산한다.

```
for (int i = 0; i < row; i++) {
for (int j = 0; j < col; j++) {
float o_r = ((float)input.at<C>(i, j)[0] - k * (float)temp.at<C>(i, j)[0]) /
(1.0f - k);
if (o_r < 0) { o_r = 0; }
if (o_r > 255) { o_r = 255; }
output.at<C>(i, j)[0] = (G)(o_r);
float o_g = ((float)input.at<C>(i, j)[1] - k * (float)temp.at<C>(i, j)[1]) /
(1.0f - k);
if (o_g < 0) { o_g = 0; }
if (o_g > 255) { o_g = 255; }
output.at<C>(i, j)[1] = (G)(o_g);
float o_b = ((float)input.at<C>(i, j)[2] - k * (float)temp.at<C>(i, j)[2]) /
(1.0f - k);
if (o_b < 0) { o_b = 0; }
if (o_b > 255) { o_b = 255; }
output.at<C>(i, j)[2] = (G)(o_b);
}
}
```

<결과분석>

output = unsharpmask(input, 3, 3, 3, k, "zero-paddle")로 호출한다. 입력 이미지로는 "lena.jpg"를 사용했다. gaussian 필터 또한 커널 크기 3, 수직 및 수평 표준편차 3으로 호출된다. 원본 이미지, 가우시안 필터링한 low-pass 이미지, unsharp-masking한 이미지를 출력했다. 관찰해보면 결과 이미지가 선명해진 것을 확인할 수 있다. 이후 동일한 k=0.5, n=sigmaT=sigmaS=3으로 경계 처리 옵션을 달리하여 3번 실행하였다.

이후 k값에 따른 sharpening 정도를 비교하기 위해 k=0.2, k=0.5, k=0.8 각각 호출하였고, k가 커질수록 high-frequency 성분이 강조되므로 sharp한 이미지가 생성된다.



[12. UnsharpRGB.cpp]

UnsharpRGB.cpp는 컬러 이미지에 대해 unsharp masking을 수행한다. 동일하게 $output = I - kL / 1 - k$ 공식을 사용하며 R,G,B 채널을 분리하고 컬러 이미지용 gaussianfilter() 함수를 사용하는 것을 제외하고 UnsharpGray.cpp와 동일하다.

<결과분석>

output = unsharpmask(input, 3, 3, 3, k, "mirroring")으로 호출한다. 입력 이미지로는 "lena.jpg"를 사용했다. gaussian 필터 또한 커널 크기 3, 수직 및 수평 표준편차 3으로 호출된다.



마찬가지로 아래와 같이 이미지 모서리 픽셀들이 밝게 표시되었다.



이후 k 값에 따른 sharpening 정도를 비교하기 위해 $k=0.1$, $k=0.5$, $k=0.8$ 로 각각 호출하였고, 그 결과는 아래와 같다. 아래 결과 사진과 같이 k 가 클수록 결과 이미지가 sharp하다.

